

Lab 6.2. Activities and Intents

1. Introduction
2. Starter code
3. Words app overview
4. Introduction to Intents
5. Set Up Explicit Intent
6. Set Up DetailActivity
7. Set Up Implicit Intent
8. Set Up Menu and Icons
9. Implement Menu button
10. Solution code
11. Summary
12. Learn more

1. Introduction

So far, the apps you've worked on have had only one activity. In reality, many Android apps require multiple activities, with navigation between them.

In this codelab, you'll build out a dictionary app so that it uses multiple activities, uses intents to navigate between them, and passes data to other apps.

Prerequisites

You should be able to:

- Navigate a project in Android Studio.
- Work with and add XML resources in Android studio.
- Override and implement methods in an existing class.
- Create instances of Kotlin classes, access class properties, and call methods.
- Refer to the documentation on developer.android.com to learn more about specific classes.

What you'll learn

How to:

- Use an explicit intent to navigate to a specific activity.

- Use an implicit intent to navigate to content in another app.
- Add menu options to add buttons to the app bar.

What you'll build

- Modify a dictionary app to implement navigation between screens using intents and adding an options menu.

What you need

- A computer with Android studio installed.

2. Starter code

On the next few steps, you'll be working on the Words app. The Words app is a simple dictionary app, with a list of letters, words for each letter, and the ability to look up definitions of each word in the browser.

Words	← Words That S
	AB
	AC
	ALPH
	ANC
	ANEC

google.com/search?q=about

[ALL](#)
[BOOKS](#)
[NEWS](#)

Showing previously fetched

Dictionary

a·bout
 /əˈbout/

preposition

- on the subject of; concerning
 "I was thinking about..."
Similar: regarding
- BRITISH**
 used to indicate movement in a particular
 area.
 "she looked about the room"
Similar: around

adverb

- BRITISH**
 used to indicate movement in a particular
 area.
 "men were floundering about"
Similar: around

There's a lot going on, but don't worry—you won't have to build an entire app just to learn about intents. Instead, you've been provided an incomplete version of the project, or starter project.

While all the screens are implemented, you can't yet navigate from one screen to the other. Your task is to use intents so that the entire project is working, without having to build everything from scratch.

Download the starter code for this codelab

This codelab provides starter code for you to extend with features taught in this codelab. Starter code may contain code that is familiar to you from previous codelabs. It may also contain code that is unfamiliar to you, and that you will learn about in later codelabs.

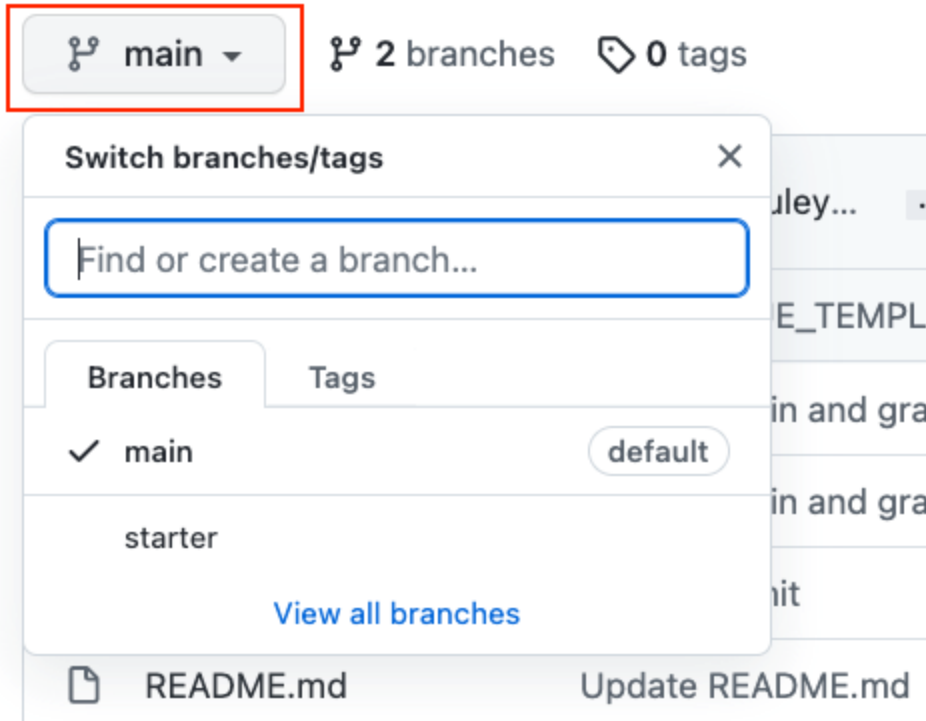
When you download the starter code from GitHub, note that the folder name is `android-basics-kotlin-words-app-starter`. Select this folder when you open the project in Android Studio.

Starter Code URL: <https://github.com/google-developer-training/android-basics-kotlin-words-app>

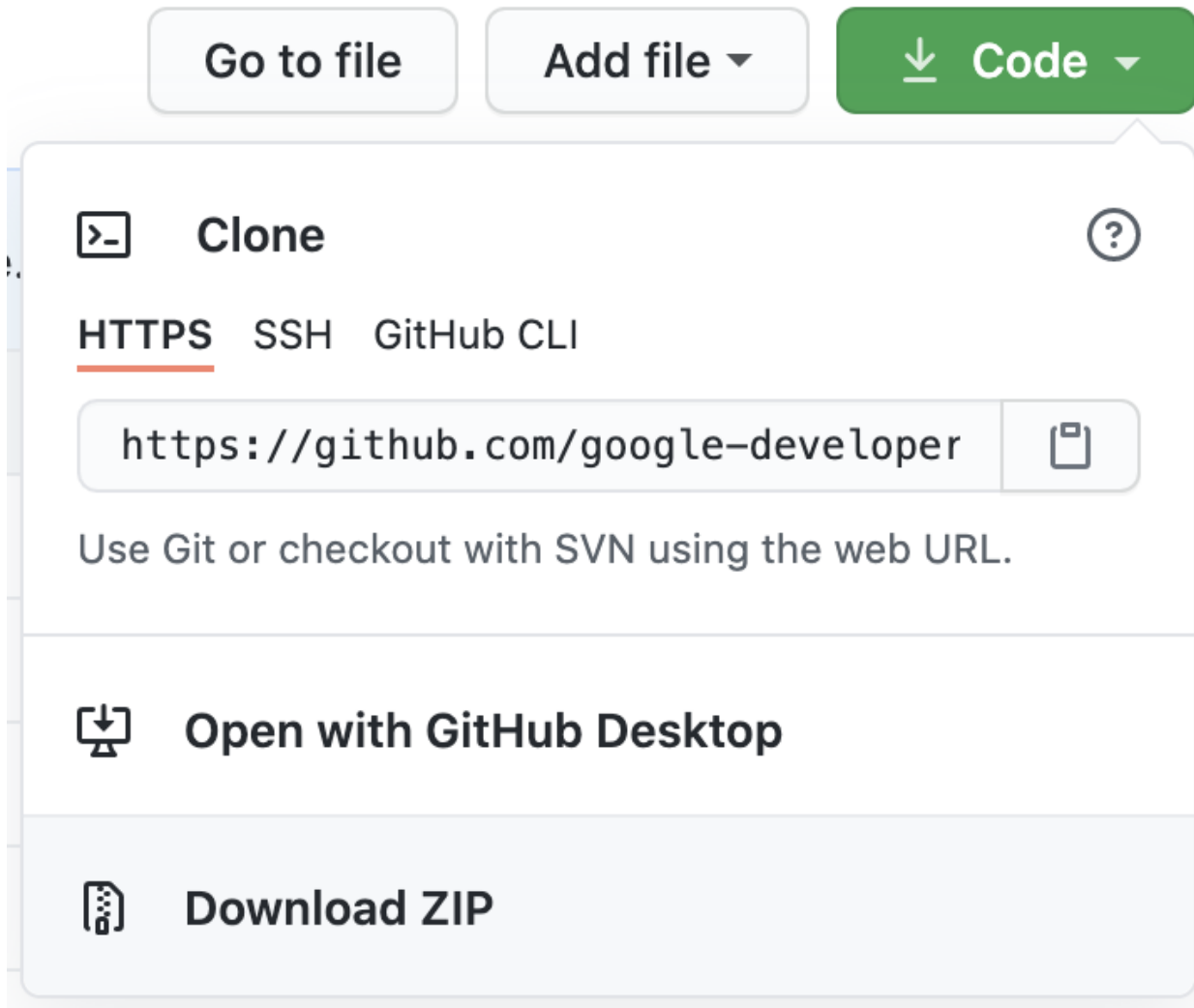
Branch name with starter code: `starter`

If you are familiar with git commands, note that the starter code is in a branch called "starter". After you clone the repo, check out the code from the `origin/starter` branch. If you haven't used git commands before, follow the below steps to download the code from GitHub.

1. Navigate to the provided GitHub repository page for the project.
2. Verify that the branch name matches the branch name specified in the codelab. For example, in the following screenshot the branch name is **main**.



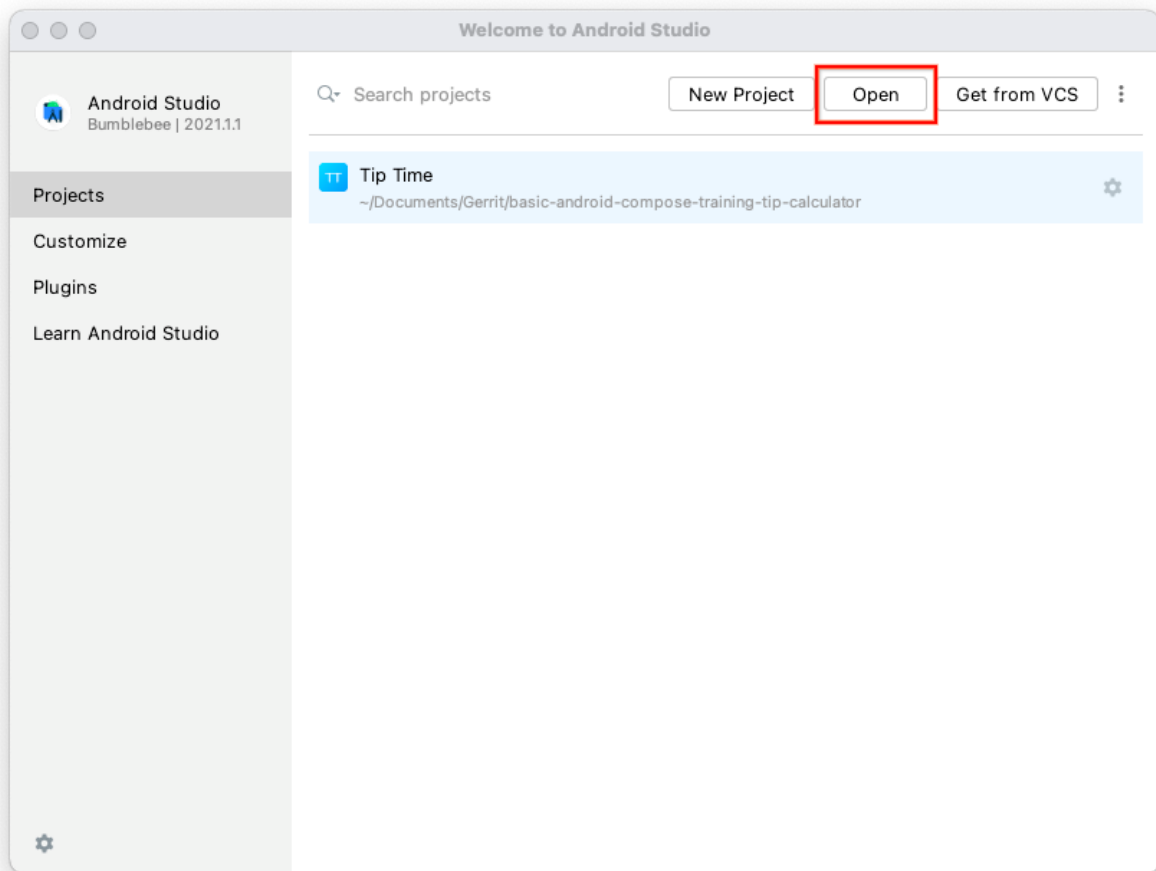
3. On the GitHub page for the project, click the **Code** button, which brings up a popup.



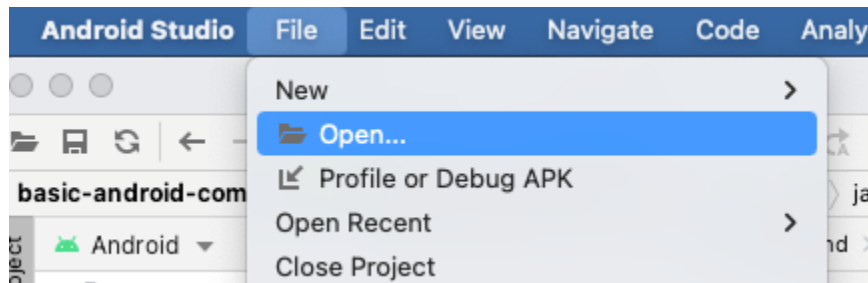
4. In the popup, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
5. Locate the file on your computer (likely in the **Downloads** folder).
6. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

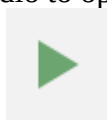
1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open**.



Note: If Android Studio is already open, instead, select the **File > Open** menu option.



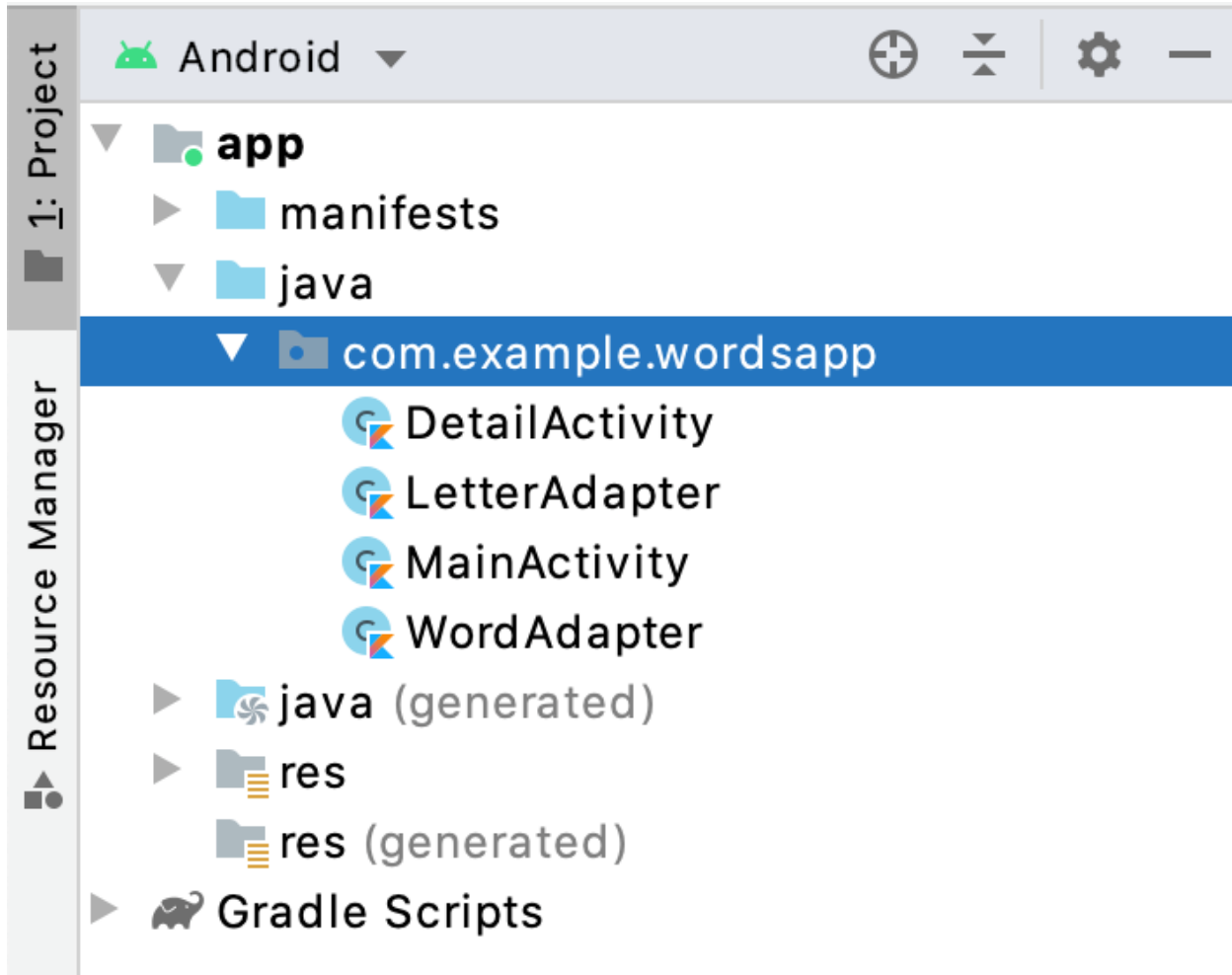
3. In the file browser, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it builds as expected.

3. Words app overview

Before moving forward, take a moment to familiarize yourself with the project. All the concepts should be familiar from the previous unit. Right now, the app consists of two activities, each with a recycler view and an adapter.



You'll specifically be working with the following files:

1. `LetterAdapter` is used by the `RecyclerView` in `MainActivity`. Each letter is a button with an `onClickListener`, which is currently empty. This is where you'll handle button presses to navigate to the `DetailActivity`.
2. `WordAdapter` is used by the `RecyclerView` in `DetailActivity` to display a list of words. While you can't navigate to this screen quite yet, just know that each word also has a corresponding button with an `onClickListener`. This is where you'll add code that navigates to the browser, to show a definition for the word.
3. `MainActivity` will also need a few changes. This is where you'll implement the options menu to display a button that allows users to toggle between list and grid layouts.

Words



A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

V

W

X

Y

Z

Once you feel comfortable with the project so far, continue to the next section where you'll learn about intents.

4. Introduction to Intents

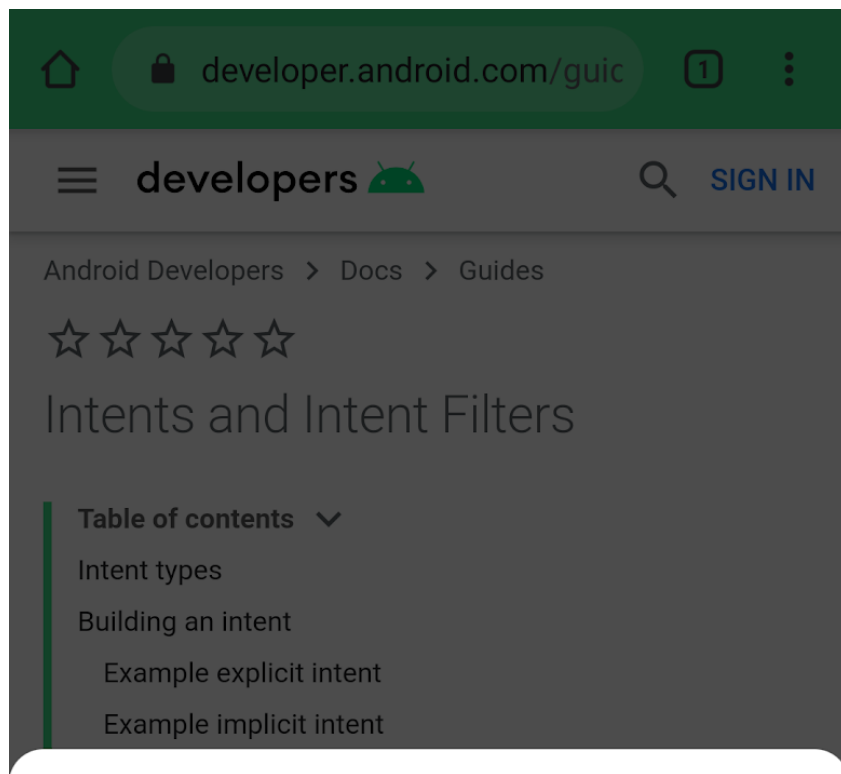
Now that you've set up the initial project, let's discuss intents and how you can use them in your app.

An *intent* is an object representing some action to be performed. The most common, but certainly not only, use for an intent is to launch an activity. There are two types of intents—**implicit** and **explicit**. An **explicit intent** is highly specific, where you know the exact activity to be launched, often a screen in your own app.

An **implicit intent** is a bit more abstract, where you tell the system the type of action, such as opening a link, composing an email, or making a phone call, and the system is responsible for figuring out how to fulfill the request. You've probably seen both kinds of intents in action without knowing it. Generally, when showing an activity in your own app, you use an explicit intent.

Words	← Words That Start With A
A	ABOUT
B	ACNE
C	ALPHABET
D	ANCHOR
E	ANECDOTE
F	
G	
H	
I	
J	
K	
L	
M	

However, for actions that don't necessarily involve the current app—say, you found an interesting Android documentation page and want to share it with friends—you'd use an **implicit intent**. You might see a menu like this asking which app to use to share the page.



Share

<https://developer.android.com/guide/components/intents-filters>



No recommended people to share with



Chrome
Print



Bluetooth

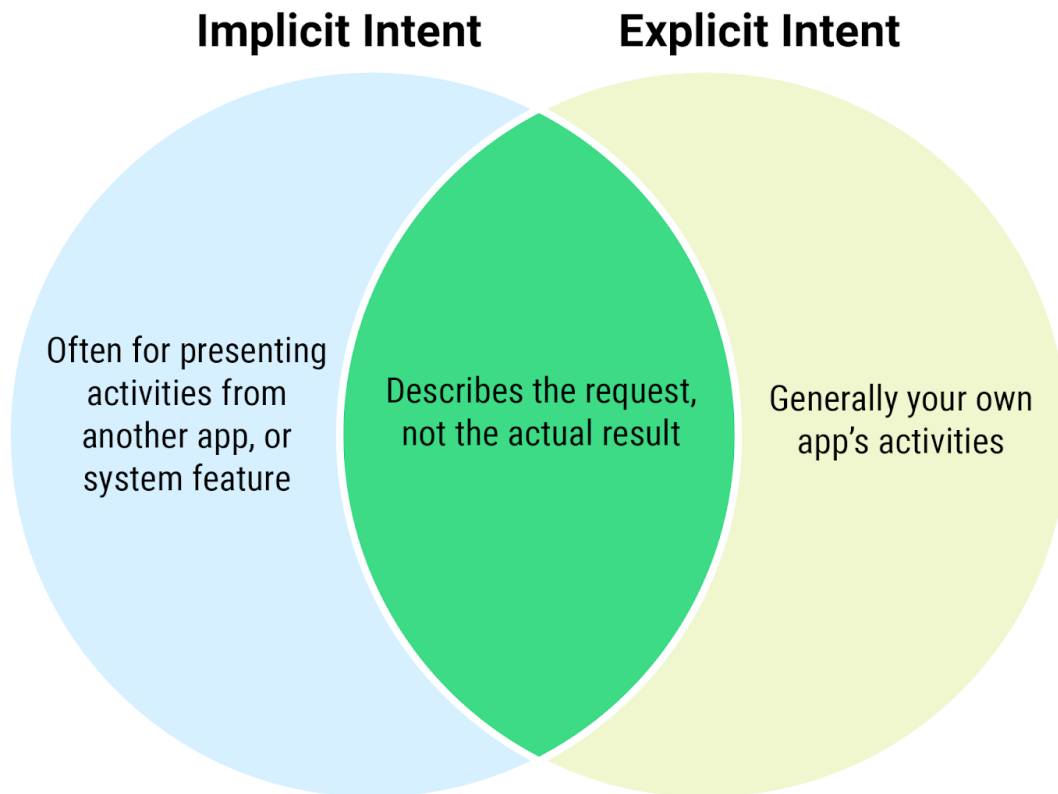


Gmail



Messages

You use an explicit intent for actions or presenting screens in your own app and are responsible for the entire process. You commonly use implicit intents for performing actions involving other apps and rely on the system to determine the end result. You'll use both types of intents in the Words app.



5. Set Up Explicit Intent

It's time to implement your first intent. On the first screen, when the user taps a letter, they should be taken to a second screen with a list of words. The `DetailActivity` is already implemented, so all that's needed is to launch it with an intent. Because your app knows exactly which activity should be launched, you use an explicit intent.

Creating and using an intent takes just a few steps:

1. Open `LetterAdapter.kt` and scroll down to `onBindViewHolder()`. Below the line to set the button text, set the `onClickListener` for `holder.button`.

```
holder.button.setOnClickListener {  
  
}
```

2. Then get a reference to the `context`.

```
val context = holder.itemView.context
```

3. Create an `Intent`, passing in the context and the class name of the destination activity.

```
val intent = Intent(context, DetailActivity::class.java)
```

The name of the activity you want to show is specified with `DetailActivity::class.java`. An actual `DetailActivity` object is created behind the scenes.

4. Call the `putExtra` method, passing in "letter" as the first argument and the button text as the second argument.

```
intent.putExtra("letter", holder.button.text.toString())
```

What's an extra? Remember that an intent is simply a set of instructions—there's no instance of the destination activity just yet. Instead, an extra is a piece of data, such as a number or string, that is given a name to be retrieved later. This is similar to passing an argument when you call a function. Because a `DetailActivity` can be shown for any letter, you need to tell it which letter to present.

Also, why do you think it's necessary to call `toString()`? The button's text is already a string, right?

Sort of. It's actually of type `CharSequence`, which is something called an *interface*. You don't need to know anything about Kotlin interfaces for now, other than it's a way to ensure a type, such as `String`, implements specific functions and properties. You can think of a `CharSequence` as a more generic representation of a string-like class. A button's `text` property could be a string, or it could be any object that is also a `CharSequence`. The `putExtra()` method, however, accepts a `String`, not just any `CharSequence`, hence the need to call `toString()`.

5. Call the `startActivity()` method on the context object, passing in the `intent`.

```
context.startActivity(intent)
```

Now run the app and try tapping a letter. The detail screen is displayed! But no matter which letter the user taps, the detail screen always shows words for the letter A. You still have some

work to do in the detail activity so that it shows words for whichever letter is passed as the `intent` extra.

6. Set Up DetailActivity

You've just created your first explicit intent! Now onto the detail screen.

In the `onCreate` method of `DetailActivity`, after the call to `setContentView`, replace the hard coded letter with code to get the `letterId` passed in from the `intent`.

```
val letterId = intent?.extras?.getString("letter").toString()
```

There's a lot going on here, so let's take a look at each part:

First, where does the `intent` property come from? It's not a property of `DetailActivity`, but rather, a property of any activity. It keeps a reference to the intent used to launch the activity.

The `extras` property is of type `Bundle`, and as you might have guessed, provides a way to access all extras passed into the intent.

Both of these properties are marked with a question mark. Why is this? The reason is that the `intent` and `extras` properties are nullable, meaning they may or may not have a value. Sometimes you may want a variable to be `null`. The `intent` property might not actually be an `Intent` (if the activity wasn't launched from an intent) and the `extras` property might not actually be a `Bundle`, but rather a value called `null`. In Kotlin, `null` means the absence of a value. The object may exist or it may be `null`. If your app tries to access a property or call a function on a `null` object, the app will crash. To safely access this value, you put a `?` after the name. If `intent` is `null`, your app won't even attempt to access the `extras` property, and if `extras` is `null`, your code won't even attempt to call `getString()`.

How do you know which properties require a question mark to ensure null safety? You can tell if the type name is followed by either a question mark or exclamation point.

intent`?.extra`

`extras` (from `get...` `Bundle?`)

The final thing to note is the actual letter is retrieved with `getString`, which returns a `String?`, so `toString()` is called to ensure it's a `String`, and not `null`.

Now when you run the app and navigate to the detail screen, you should see the list of words for each letter.



Words That Start With A

ABOUT

ACNE

ALPHABET

ANCHOR

ANECDOTE

Cleaning Up

Both the code to perform the intent, and retrieve the selected letter hardcode the name of the `extra`, "letter". While this works for this small example, it's not the best approach for large apps where you have many more intent extras to keep track of.

While you could just create a constant called "letter", this could get unwieldy as you add more intent extras to your app. And in which class would you put this constant? Remember that the string is used in both `DetailActivity` and `MainActivity`. You need a way to define a constant that can be used across multiple classes, while keeping your code organized.

Thankfully, there's a handy Kotlin feature that can be used to separate your constants and make them usable without a particular instance of the class called [companion objects](#). A companion object is similar to other objects, such as instances of a class. However, only a single instance of a companion object will exist for the duration of your program, which is why this is sometimes called the [singleton pattern](#). While there are numerous use cases for *singletons* beyond the scope of this codelab, for now, you'll use a companion object as a way to organize constants and make them accessible outside of the `DetailActivity`. You'll start by using a companion object to refactor the code for the "letter" extra.

1. In `DetailActivity`, just above `onCreate`, add the following:

```
companion object {  
  
}
```

Notice how this is similar to defining a class, except you use the `object` keyword. There's also a keyword `companion`, meaning it's associated with the `DetailActivity` class, and we don't need to give it a separate type name.

2. Within the curly braces, add a property for the letter constant.

```
const val LETTER = "letter"
```

3. To use the new constant, update your hard coded letter call in `onCreate()` as follows:

```
val letterId = intent?.extras?.getString(LETTER).toString()
```

Again, notice that you reference it with dot notation as usual, but the constant belongs to `DetailActivity`.

4. Switch over to `LetterAdapter`, and modify the call to `putExtra` to use the new constant.

```
intent.putExtra(DetailActivity.LETTER, holder.button.text.toString())
```

All set! By refactoring, you just made your code easier to read, and easier to maintain. If this, or any other constant you add, ever needs to change, you only need to do so in one place.

To learn more about companion objects, check out the Kotlin documentation on [Object Expressions and Declarations](#).

7. Set Up Implicit Intent

In most cases, you'll present specific activities from your own app. However, there are some situations where you may not know which activity, or which app, you want to launch. On our detail screen, each word is a button that will show the user definition of the word.

For our example, you're going to use the dictionary functionality provided by a Google search. Instead of adding a new activity to your app, however, you're going to launch the device's browser to show the search page.

So perhaps you'd need an intent to load the page in Chrome, the default browser on Android?

Not quite.

It's possible that some users prefer a third party browser. Or their phone comes with a browser preinstalled by the manufacturer. Perhaps they have the Google search app installed—or even a third-party dictionary app.

You can't know for sure what apps the user has installed. Nor can you assume how they may want to look up a word. This is a perfect example of when to use an implicit intent. Your app provides information to the system on what the action should be, and the system figures out what to do with that action, prompting the user for any additional information as needed.

Do the following to create the implicit intent:

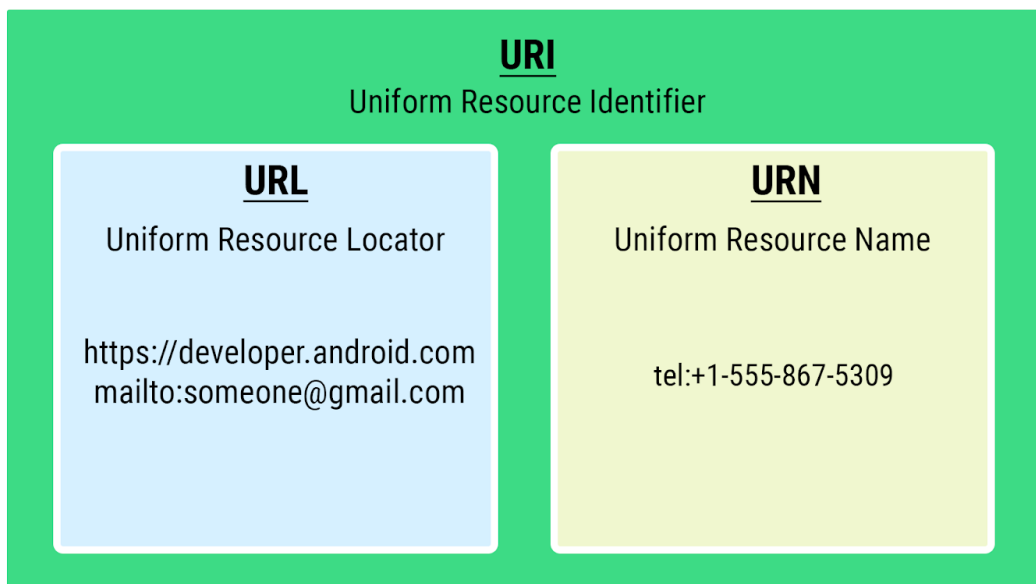
1. For this app, you'll perform a Google search for the word. The first search result will be a dictionary definition of the word. Since the same base URL is used for every search, it's a good idea to define this as its own constant. In `DetailActivity`, modify the companion object to add a new constant, `SEARCH_PREFIX`. This is the base URL for a Google search.

```
companion object {  
    const val LETTER = "letter"  
    const val SEARCH_PREFIX = "https://www.google.com/search?q="  
}
```

2. Then, open up `WordAdapter` and in the `onBindViewHolder()` method, call `setOnClickListener()` on the button. Start by creating a `Uri` for the search query. When calling `parse()` to create a `Uri` from a `String`, you need to use string formatting so that the word is appended to the `SEARCH_PREFIX`.

```
holder.button.setOnClickListener {  
    val queryUrl: Uri = Uri.parse("${DetailActivity.SEARCH_PREFIX}${item}")  
}
```

If you're wondering what a *URI* is, it's not a typo, but stands for *Uniform Resource Identifier*. You may already know that a URL, or *Uniform Resource Locator*, is a string that points to a webpage. A URI is a more general term for the format. All URLs are URIs, but not all URIs are URLs. Other URIs, for example, an address for a phone number, would begin with `tel:`, but this is considered a URN or *Uniform Resource Name*, rather than a URL. The data type used to represent both is called `URI`.



Notice how there's no reference to any activity in your own app here. You simply provide a `URI`, without an indication of how it's ultimately used.

2. After defining `queryUrl`, initialize a new `intent` object:

```
val intent = Intent(Intent.ACTION_VIEW, queryUrl)
```

Instead of passing in a context and an activity, you pass in `Intent.ACTION_VIEW` along with the URI.

`ACTION_VIEW` is a generic intent that takes a URI, in your case, a web address. The system then knows to process this intent by opening the URI in the user's web browser. Some other intent types include:

- `CATEGORY_APP_MAPS` – launching the maps app
- `CATEGORY_APP_EMAIL` – launching the email app
- `CATEGORY_APP_GALLERY` – launching the gallery (photos) app
- `ACTION_SET_ALARM` – setting an alarm in the background
- `ACTION_DIAL` – initiating a phone call

To learn more, visit the documentation for some [commonly used intents](#).

3. Finally, even though you aren't launching any particular activity in your app, you're telling the system to launch another app, by calling `startActivity()` and pass in the `intent`.

```
context.startActivity(intent)
```

Now when you launch the app, navigate to the words list, and tap one of the words, your device should navigate to the URL (or present a list of options depending on your installed apps).

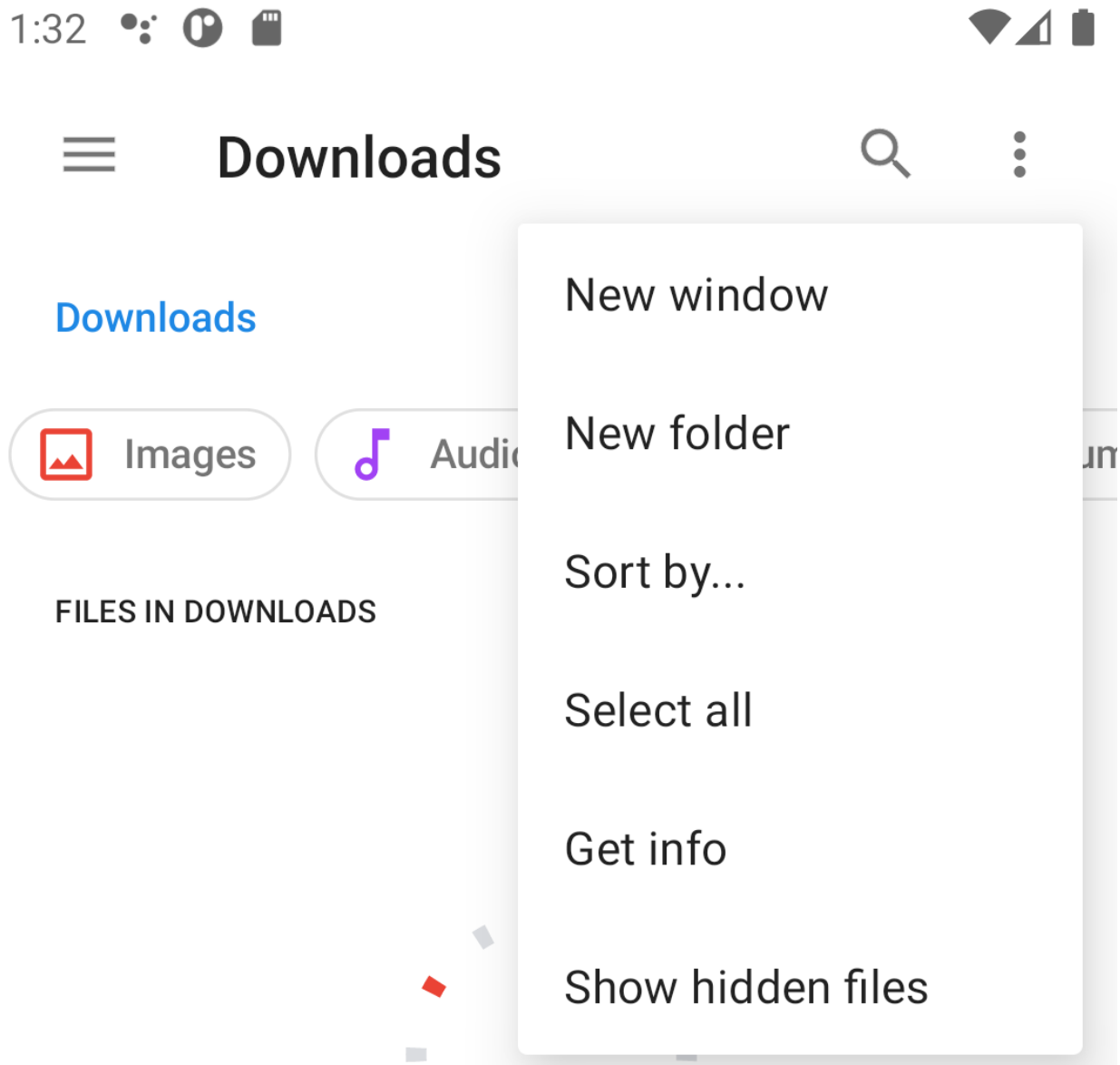
The exact behavior will differ among users, providing a seamless experience for everyone, without complicating your code.

8. Set Up Menu and Icons

Now that you've made your app fully navigable by adding explicit and implicit intents, it's time to add a menu option so that the user can toggle between list and grid layouts for the letters.

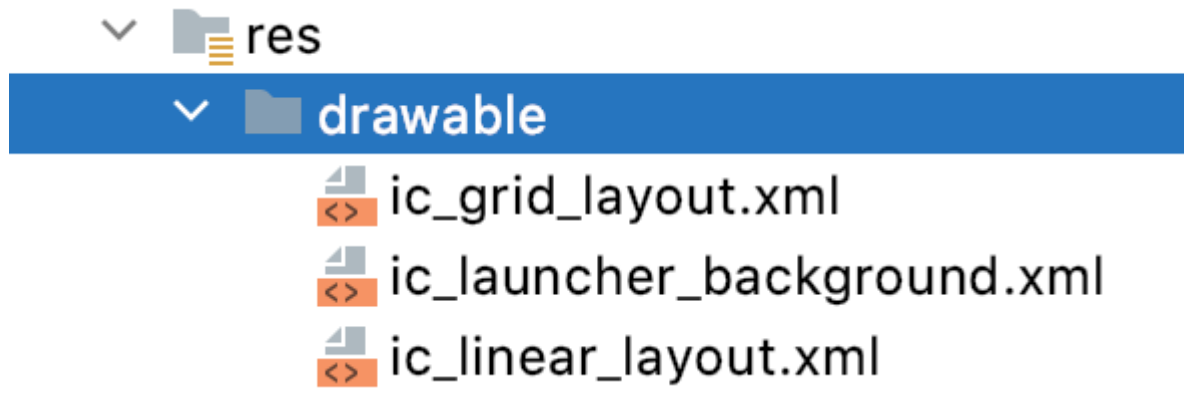
Words		Words		
A		A	B	C
B		E	F	G
C		I	J	K
D		M	N	O
E		Q	R	S
F		U	V	W
G		Y	Z	
H				
I				
J				
K				
L				
M				

By now, you've probably noticed many apps have this bar at the top of the screen. This is called the app bar, and in addition to showing the app's name, the app bar can be customized and host lots of useful functionality, like shortcuts for useful actions, or overflow menus.

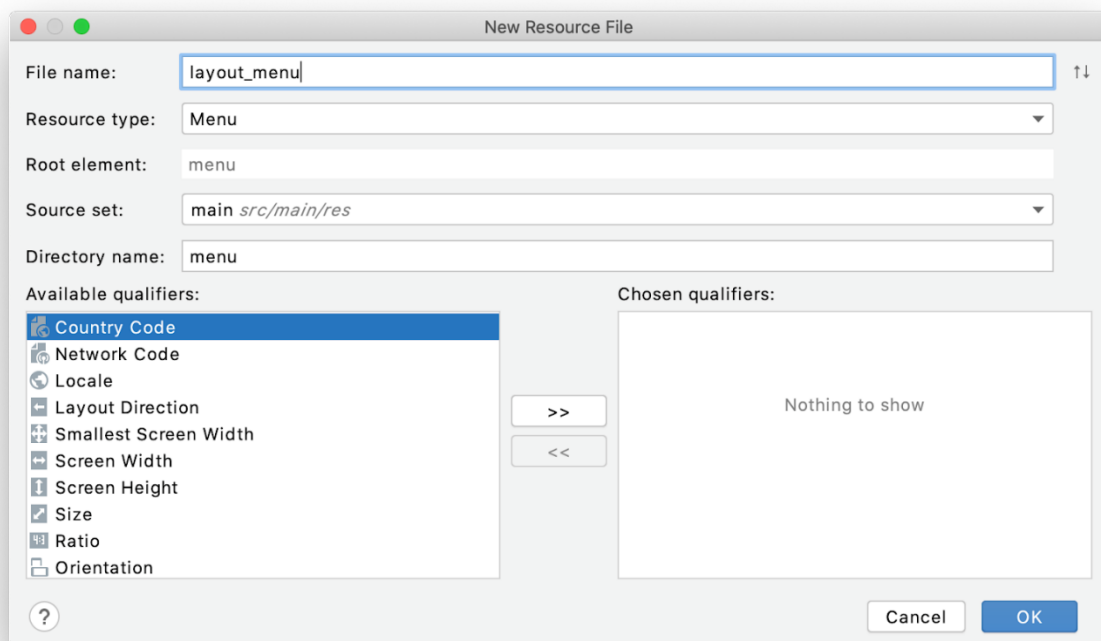


For this app, while we won't add a fully fledged menu, you'll learn how to add a custom button to the app bar, so that the user can change the layout.

1. First, you need to import two icons to represent the grid and list views. Add the clip art vector assets called "view module" (name it **ic_grid_layout**) and "view list" (name it **ic_linear_layout**). If you need a refresher on adding material icons, take a look at the instructions on [this page](#).



2. You also need a way to tell the system what options are displayed in the app bar, and which icons to use. To do this, add a new resource file by right-clicking on the **res** folder and selecting **New > Android Resource File**. Set the **Resource type** to **Menu** and the **File name** to `layout_menu`.



3. Click **OK**.
4. Open **res/Menu/layout_menu**. Replace the contents of `layout_menu.xml` with the following:

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">
    <item android:id="@+id/action_switch_layout">
```

```

        android:title="@string/action_switch_layout"
        android:icon="@drawable/ic_linear_layout"
        app:showAsAction="always" />
    </menu>

```

The structure of the menu file is fairly simple. Just like a layout starts with a layout manager to hold individual views, a menu xml file starts with a menu tag, which contains individual options.

Your menu just has one button, with a few properties:

- **id:** Just like views, the menu option has an id so that it can be referenced in code.
- **title:** This text won't actually be visible in your case, but may be useful for screen readers to identify the menu
- **icon:** The default is `ic_linear_layout`. However, this will be toggled on and off to show the grid icon, when the button is selected.
- **showAsAction:** This tells the system how to show the button. Because it's set to `always`, this button will always be visible in the app bar, and not become part of an overflow menu.

Of course, just having the properties set doesn't mean the menu will actually do anything.

You'll still need to add some code in `MainActivity.kt` to get the menu working.

9. Implement Menu button

To see your menu button in action, there are a few things to do in `MainActivity.kt`.

1. First, it's a good idea to create a property to keep track of which layout state the app is in. That will make it easier to toggle the layout button. Set the default value to `true`, as the linear layout manager will be used by default.

```
private var isLinearLayoutManager = true
```

2. When the user toggles the button, you want the list of items to turn into a grid of items. If you recall from learning about recycler views, there are many different layout managers, one of which, `GridLayoutManager` allows for multiple items on a single row.

```

private fun chooseLayout() {
    if (isLinearLayoutManager) {
        recyclerView.layoutManager = LinearLayoutManager(this)
    } else {
        recyclerView.layoutManager = GridLayoutManager(this, 4)
    }
    recyclerView.adapter = LetterAdapter()
}

```

Here you use an `if` statement to assign the layout manager. In addition to setting the `layoutManager`, this code also assigns the adapter. `LetterAdapter` is used for both list and grid layouts.

3. When you initially set up the menu in xml, you gave it a static icon. However, after toggling the layout, you should update the icon to reflect its new function—switching back to the list layout. Here, simply set the linear and grid layout icons, based on the layout the button will switch back to, next time it's tapped.

```
private fun setIcon(menuItem: MenuItem?) {
    if (menuItem == null)
        return

    // Set the drawable for the menu icon based on which LayoutManager is
    // currently in use

    // An if-clause can be used on the right side of an assignment if all
    // paths return a value.
    // The following code is equivalent to
    // if (isLinearLayoutManager)
    //     menu.icon = ContextCompat.getDrawable(this,
    R.drawable.ic_grid_layout)
    // else menu.icon = ContextCompat.getDrawable(this,
    R.drawable.ic_linear_layout)
    menuItem.icon =
        if (isLinearLayoutManager)
            ContextCompat.getDrawable(this, R.drawable.ic_grid_layout)
        else ContextCompat.getDrawable(this, R.drawable.ic_linear_layout)
}
```

The icon is conditionally set based on the `isLinearLayoutManager` property.

For your app to actually use the menu, you need to override two more methods.

- `onCreateOptionsMenu`: where you inflate the options menu and perform any additional setup
- `onOptionsItemSelected`: where you'll actually call `chooseLayout()` when the button is selected.

1. **Override `onCreateOptionsMenu` as follows:**

```
override fun onCreateOptionsMenu(menu: Menu?): Boolean {
    menuInflater.inflate(R.menu.layout_menu, menu)

    val layoutButton = menu?.findItem(R.id.action_switch_layout)
    // Calls code to set the icon based on the LinearLayoutManager of the
    RecyclerView
    setIcon(layoutButton)

    return true
}
```

Nothing fancy here. After inflating the layout, you call `setIcon()` to ensure the icon is correct, based on the layout. The method returns a `Boolean`—you return `true` here since you want the options menu to be created.

2. Implement as shown `onOptionsItemSelected` with just a few more lines of code.

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {
    return when (item.itemId) {
        R.id.action_switch_layout -> {
            // Sets isLinearLayoutManager (a Boolean) to the opposite value
            isLinearLayoutManager = !isLinearLayoutManager
            // Sets layout and icon
            chooseLayout()
            setIcon(item)

            return true
        }
        // Otherwise, do nothing and use the core event handling

        // when clauses require that all possible paths be accounted for
        explicitly,
        // for instance both the true and false cases if the value is a
        Boolean,
        // or an else to catch all unhandled cases.
        else -> super.onOptionsItemSelected(item)
    }
}
```

This is called any time a menu item is tapped so you need to be sure to check which menu item is tapped. You use a `when` statement, above. If the `id` matches the `action_switch_layout` menu item, you negate the value of `isLinearLayoutManager`. Then, call `chooseLayout()` and `setIcon()` to update the UI accordingly.

One more thing, before you run the app. Since the layout manager and adapter are now set in `chooseLayout()`, you should replace that code in `onCreate()` to call your new method. `onCreate()` should look like the following after the change.

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    val binding = ActivityMainBinding.inflate(layoutInflater)
    setContentView(binding.root)

    recyclerView = binding.recyclerView
    // Sets the LinearLayoutManager of the recyclerview
    chooseLayout()
}
```

Now run your app and you should be able to toggle between list and grid views using the menu button.

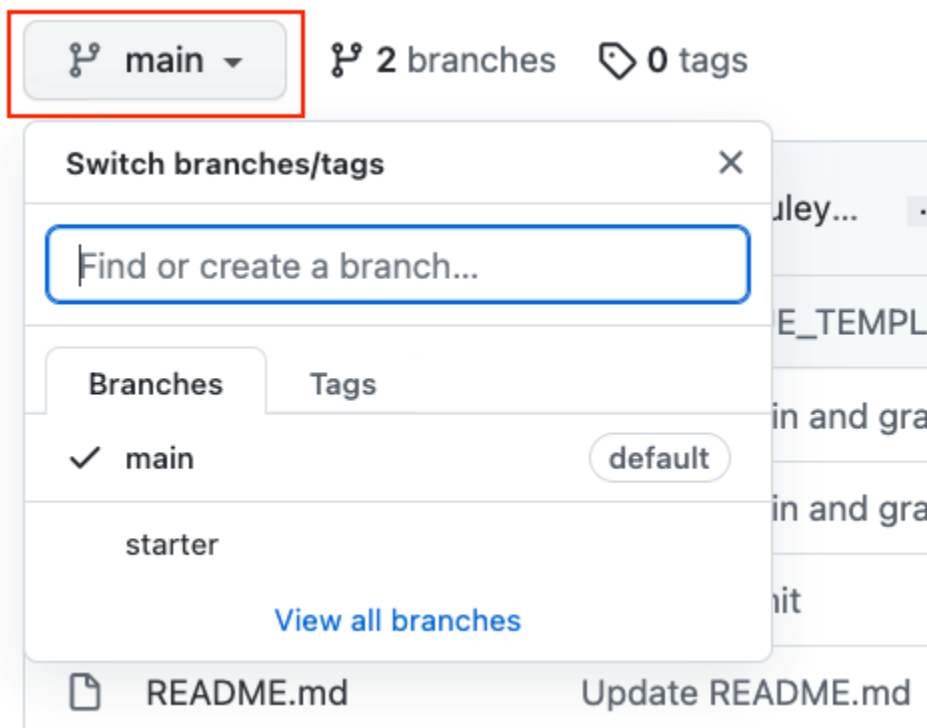
10. Solution code

The solution code for this codelab is in the project below:

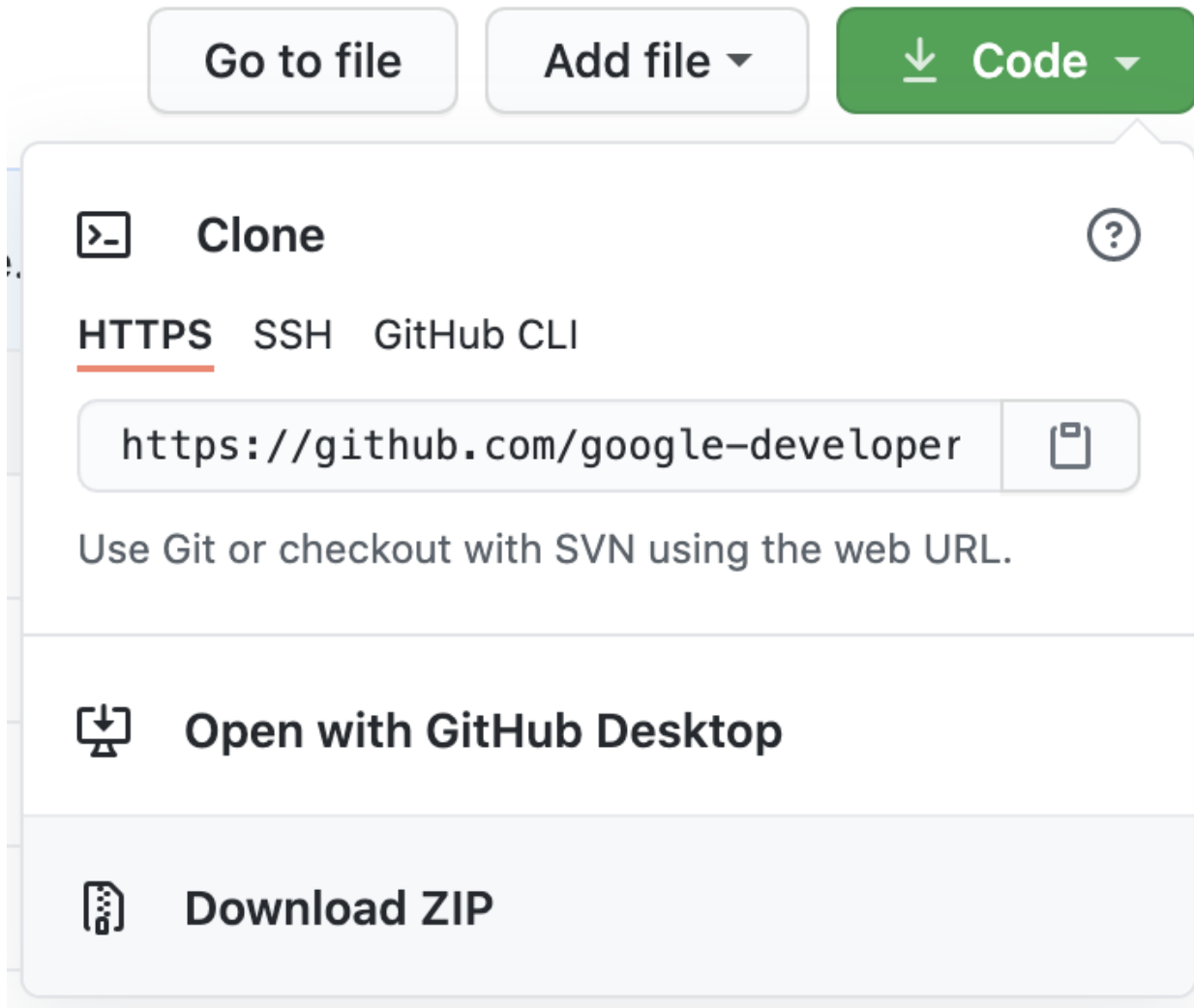
Solution Code URL: <https://github.com/google-developer-training/android-basics-kotlin-words-app>

Branch name with solution code: `activities`

1. Navigate to the provided GitHub repository page for the project.
2. Verify that the branch name matches the branch name specified in the codelab. For example, in the following screenshot the branch name is **main**.



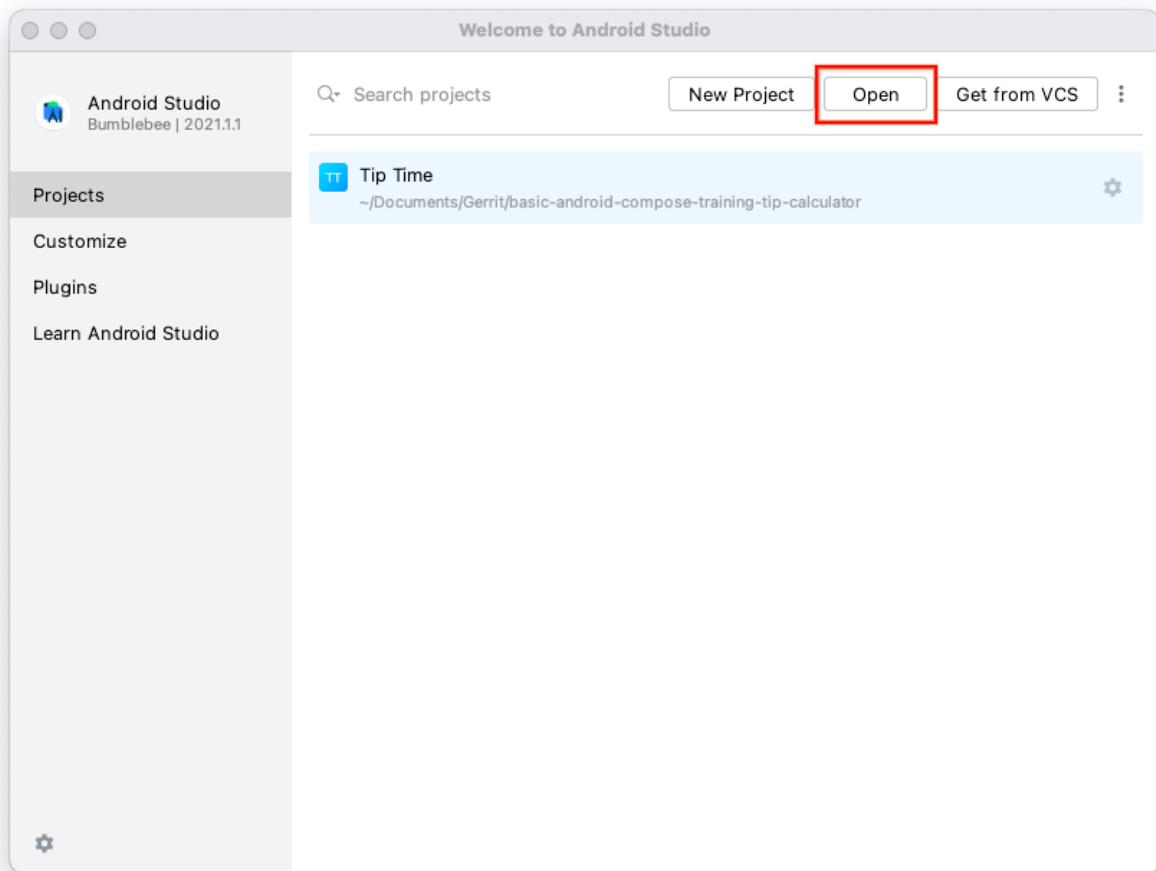
3. On the GitHub page for the project, click the **Code** button, which brings up a popup.



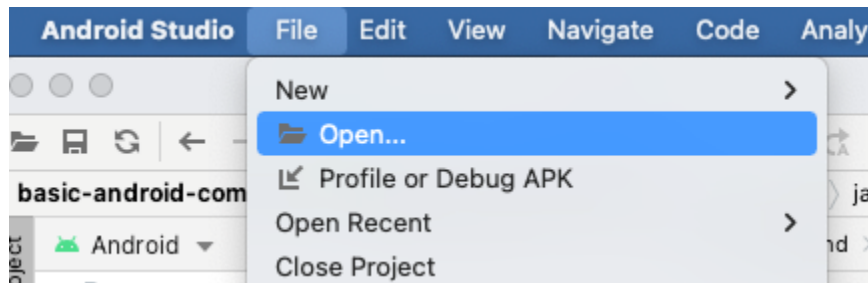
4. In the popup, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
5. Locate the file on your computer (likely in the **Downloads** folder).
6. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

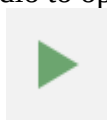
1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open**.



Note: If Android Studio is already open, instead, select the **File > Open** menu option.



3. In the file browser, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it builds as expected.

11. Summary

- Explicit intents are used to navigate to specific activities in your app.
- Implicit intents correspond to specific actions (like opening a link, or sharing an image) and let the system determine how to fulfill the intent.
- Menu options allow you to add buttons and menus to the app bar.
- Companion objects provide a way to associate reusable constants with a type, rather than an instance of that type.

To perform an intent:

- Get a reference to the context.
- Create an `Intent` object providing either an activity or intent type (depending on whether it's explicit or implicit).
- Pass any needed data by calling `putExtra()`.
- Call `startActivity()` passing in the `intent` object.

12. Learn more

- [Intents and Intent Filters](#)
- [Commonly used intents](#)
- [Creating Options Menus with XML](#)
- [Interfaces in Kotlin](#)
- [CharSequence interface](#)
- [Null Safety in Kotlin](#)
- [Object Expressions and Declarations in Kotlin](#)
- [Singleton Pattern](#)
- [Control Flow in Kotlin](#)